AI Assisted Self Healing SDN

Current demonstration results explained simply

BCSE308P Computer Networks Lab
Fall Semester 2026–27
24BCE2052
Vaibhav Bakshi

What this project demonstrates

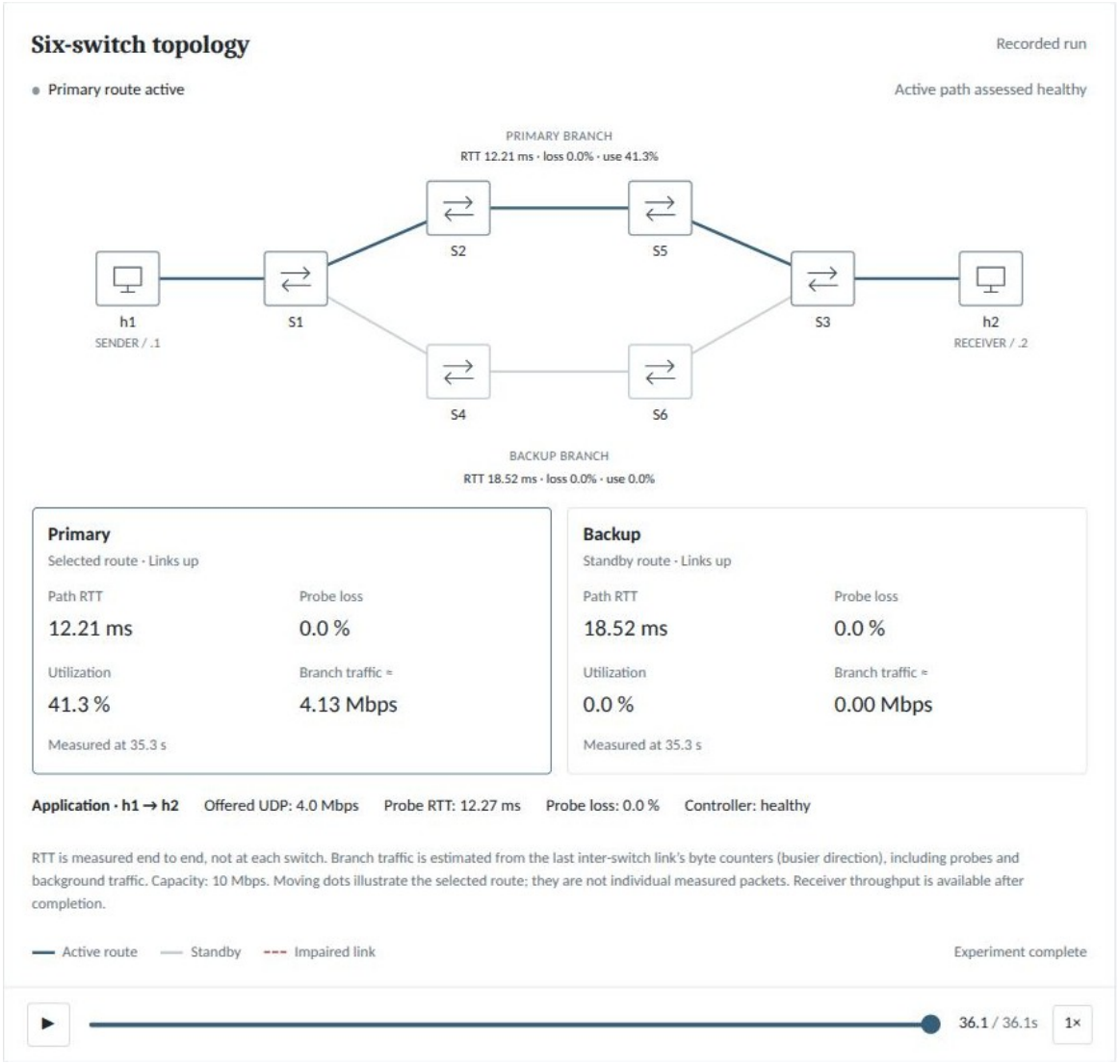
Two network paths connect the sender and receiver. The application starts on the primary path.

The controller checks both paths. If the selected path fails and the other is healthy, it changes the forwarding rules.

We test whether communication returns while the injected fault is still present.

Rerouting needs a usable backup. The system cannot repair a disconnected physical link.

Reading the network diagram



Blue means selected

The application uses the blue branch. Its card shows measured round trip time, probe loss and estimated traffic.

Grey means standby

The controller still sends small test probes through the backup. That is why an unused application route has an RTT reading.

The conditions used for these results

Setting	Value
Network	Six switches with primary and backup branches
Application traffic	4 Mbps offered to a 10 Mbps branch
Traffic duration	36 seconds, plus setup and cleanup
Fault schedule	Requested around 10 to 26 seconds
Evidence set	Six default scenarios plus two detector comparisons

Eight runs, one per condition, at 10 Mbps capacity. New runs can set capacity independently; these saved results are unchanged.

The three detection methods

Method	How it decides
RF only	A trained random forest judges the measurements.
RF + loss guard	The same forest plus a fixed rule for loss of at least 15%.
Fixed rules	Limits on loss, RTT, missing replies and utilization.

All methods share the same forwarding and recovery policy. Hard link failures use a separate link-state check.

RF + loss guard is the default shown in the scenario results.

Route change and recovery are different

2.10 seconds to change route

After the link failed, the controller selected and installed the backup route. This records the forwarding action.

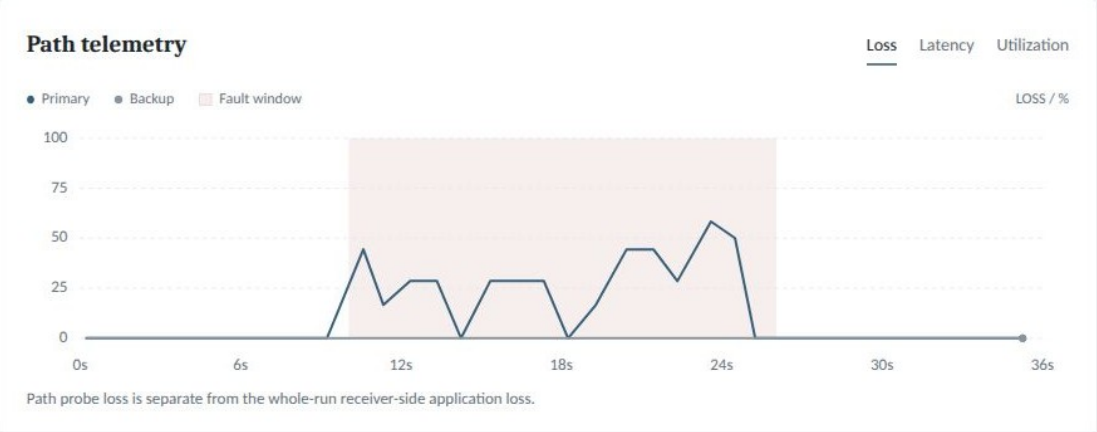
8.02 seconds to confirm recovery

The runner then observed two healthy application probe windows while the fault remained active. Probe timing and timeouts add to this delay.

Example from the current link-failure run. Both times start at actual fault onset.

Reading loss and latency

Loss in percent



Round trip time in milliseconds



Loss asks whether test packets returned. The primary can still show loss after the application moves to the backup.

Latency asks how long a reply took. It measures the complete round trip. A missing reply does not mean zero delay.

Pink shading marks the fault window. Lines connect sampled observations.

Utilization and application load



Measured link use

Counter rate ÷ capacity gives utilization.

4.11 Mbps ÷ 10 Mbps = 41.1%.

Capacity applies to both branches. Utilization uses the chosen capacity. At 20 Mbps, 4 Mbps is approximately 20% utilization before other traffic.

Set link capacity in the dashboard or use --capacity 20 --load 4. Default: 10 Mbps; maximum: 100 Mbps. Congestion background = 1.2 × capacity.

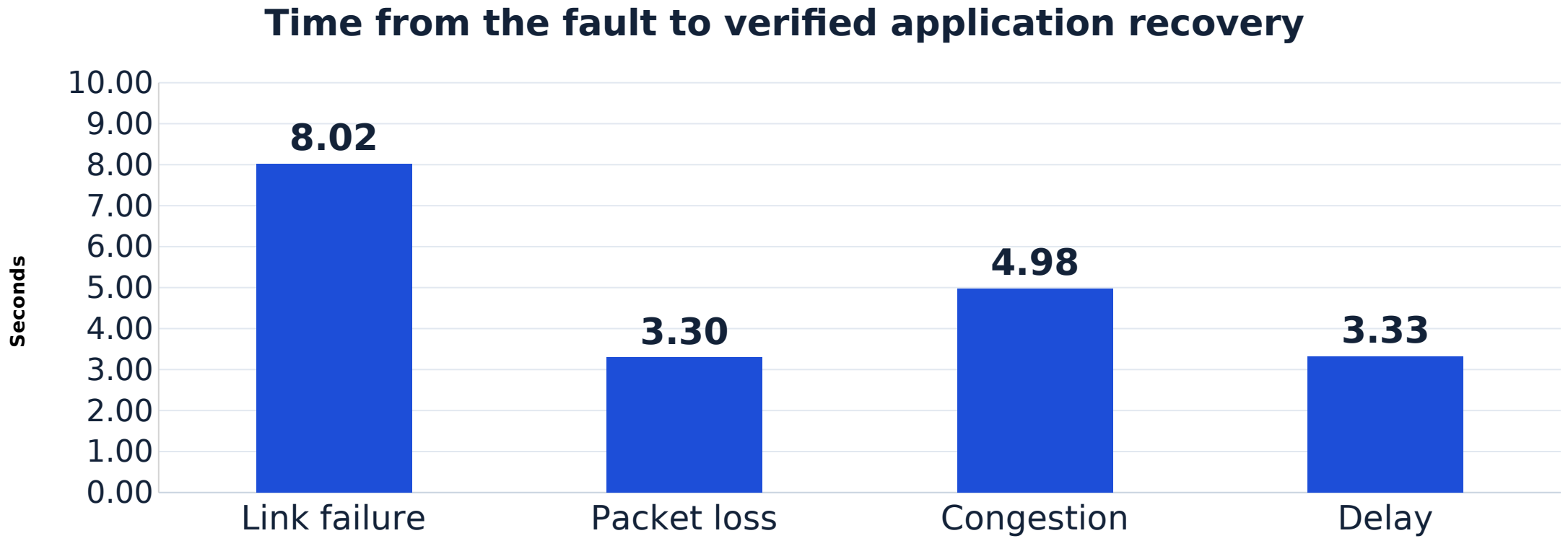
What happened in each scenario

Scenario	Data received	Packets lost	What happened
Normal	4.00 Mbps	0.00%	No fault to recover from
Link failure	3.76 Mbps	5.86%	Restored in 8.02 s
Packet loss	3.97 Mbps	0.64%	Restored in 3.30 s
Congestion	3.89 Mbps	2.81%	Restored in 4.98 s
Increased delay	4.00 Mbps	0.01%	Restored in 3.33 s
Both paths down	2.00 Mbps	50.02%	No healthy alternative

Default RF + loss guard, six switches, 4 Mbps offered traffic. Each row is one current 36-second run.

“Restored” means two healthy application probe windows after rerouting while the fault remains active.

How long did recovery confirmation take



A shorter bar means earlier confirmation in that run. Link failure took about 8 seconds including probe waits and confirmation.

Normal needs no recovery; both paths down has no reroute recovery. Neither means zero seconds.

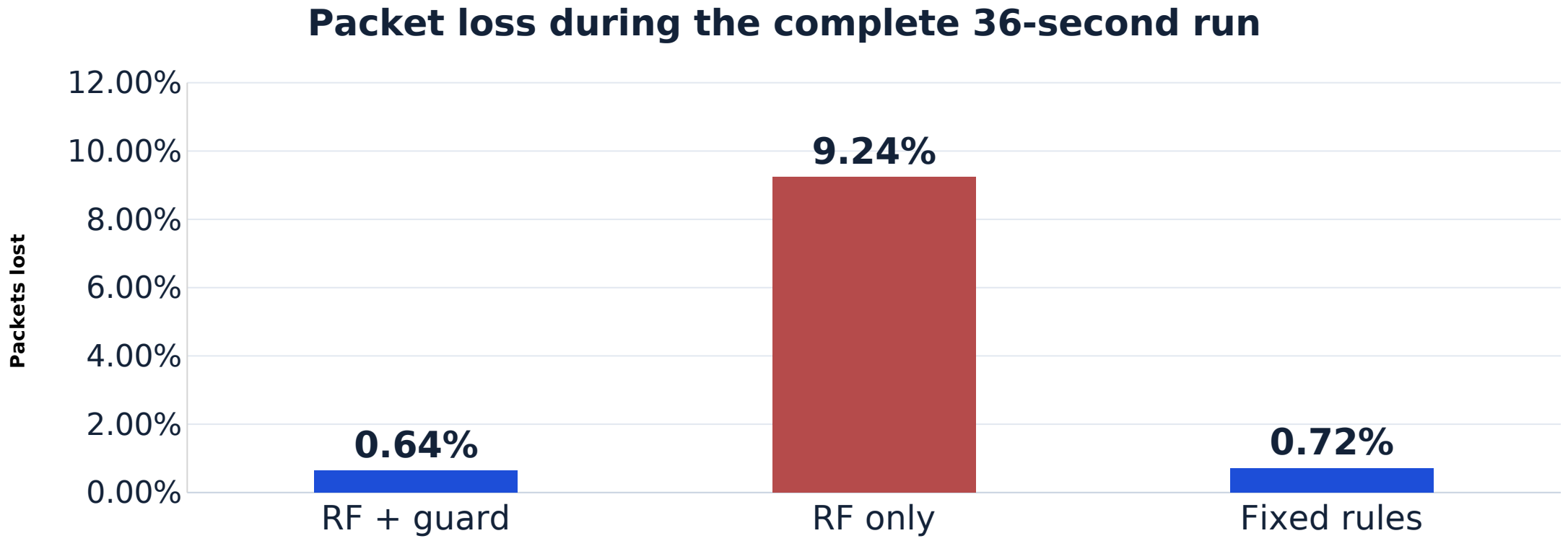
Which method handled packet loss

Packet-loss test	RF + guard	RF only	Fixed rules
Changed route?	Yes	No	Yes
Confirmed recovery	3.30 s	Not observed	3.35 s
Data received	3.97 Mbps	3.63 Mbps	3.97 Mbps
Packets lost	0.64%	9.24%	0.72%

The same packet-loss scenario ran once with each method. Both guarded RF and fixed rules restored communication. RF alone missed it.

The guard helps in this test. A small timing difference between successful methods does not prove that one is consistently faster.

How much application traffic was lost



RF only lost about 9.24% of application packets in this run. Guarded RF lost 0.64%, and fixed rules lost 0.72%.

Receiver-side UDP loss differs from primary-path ping loss.

Why RF needs safeguards

Question	What the current implementation does
What does RF read?	Loss, round trip time, RTT variation and utilization
What does the guard add?	It declares measured loss of 15% or more unhealthy.
What handles a broken link?	A separate link-state check
How reliable is the model?	Stored validation covers its original lab data only.
What did the live test expose?	RF alone missed the packet-loss fault.

The stored model report says 100% on 544 validation rows. This is not a guarantee for new faults or networks.

The 0.65 decision threshold does not mean 65% accuracy. A guarded success cannot be credited entirely to RF.

What the current tests establish

Guarded RF restored communication in the link failure, packet loss, congestion and delay demonstrations.

RF alone missed packet loss. The loss guard and fixed rules both recovered in that comparison.

When both paths failed, there was no healthy alternative. These are single-run observations on this testbed.